1. Introduction to Cloud Computing

Aim:

To demonstrate the fundamentals of cloud computing by uploading and retrieving files from **AWS S3** using a Python script.

Procedure:

1. Set up an **AWS S3 Bucket** with necessary permissions.
2. Create an **IAM User** with S3 access and obtain access keys.
3. Use the AWS SDK (Boto3) in a Python script to upload and download files.
4. Verify file upload and retrieval from the S3 bucket.

Souce Code:

```
import boto3
import os
# AWS S3 Configuration
bucket_name = 'your-bucket-name'
file_name = 'sample.txt'
s3_key = 'uploaded-sample.txt'
# AWS Authentication
s3 = boto3.client(
    's3',
    aws_access_key_id='YOUR_ACCESS_KEY',
    aws_secret_access_key='YOUR_SECRET_KEY',
    region_name='us-east-1'
)
# Upload File
def upload_file():
    s3.upload_file(file_name, bucket_name, s3_key)
    print(f"{file_name} uploaded to {bucket_name}")

# Download File
```

```
def download_file():
    s3.download_file(bucket_name, s3_key, f"downloaded-{file_name}")
    print(f"File downloaded as downloaded-{file_name}")
upload_file()
download_file()
```

Output:

- Running the script uploads sample.txt to AWS S3.
- The file can be accessed and downloaded from the S3 bucket

Result:

Successfully demonstrated cloud storage using AWS S3, highlighting cloud computing's flexibility and scalability.

2.Creating a Warehouse Application in Salesforce.com

Aim:

To create a warehouse management application in Salesforce using Apex to handle inventory, stock management, and order processing.

Procedure:

Step 1: Log in to Salesforce.com and navigate to the Developer Console.

Step 2: Create a new Warehouse application using standard objects.

Step 3: Configure data fields and relationships.

Step 4: Implement workflows and validation rules.

Step 5: Deploy the application and test its functionalities.

Source code:

```
// Apex Class: WarehouseManagement.cls
public class WarehouseManagement {
    public static void updateStock(String productName, Integer quantity) {
        Product__c product = [SELECT Id, Stock_Quantity__c FROM Product__c WHERE Name =
:productName LIMIT 1];
        product.Stock_Quantity__c += quantity;
        update product;
    }
```

```
    public static void createOrder(String productName, Integer quantity) {
        Product__c product = [SELECT Id, Stock_Quantity__c FROM Product__c WHERE Name =
:productName LIMIT 1];
        if (product.Stock_Quantity__c >= quantity) {
            product.Stock_Quantity__c -= quantity;
            update product;
            Order__c newOrder = new Order__c(Product_Name__c = productName, Quantity__c =
quantity);
            insert newOrder;
        } else {
            System.debug('Insufficient stock');
        }
    }
}

// Apex Trigger: StockValidation.trigger
trigger StockValidation on Order__c (before insert) {
    for (Order__c order : Trigger.new) {
        Product__c product = [SELECT Stock_Quantity__c FROM Product__c WHERE Name =
:order.Product_Name__c LIMIT 1];
        if (order.Quantity__c > product.Stock_Quantity__c) {
            order.addError('Not enough stock available');
        }
    }
}
```

Output:

Create orders while automatically updating stock levels.

Trigger validation prevents orders if stock is insufficient.

Inventory is accurately managed in Salesforce.

Result:

Successfully developed a warehouse management application using Apex in Salesforce, enabling efficient inventory and order handling.

3. Creating an Application in Salesforce.com using Apex

Aim:

To create a custom application in Salesforce.com using Apex to automate business processes and enhance CRM functionalities.

Procedure:

Step 1: Open Salesforce Developer Console.

Step 2: Write Apex code to create a custom application.

Step 3: Use Apex triggers to automate processes.

Step4: Integrate with Visualforce pages or Lightning Components.

Step 5: Deploy and test the application.

Source code:

```
// Apex Class: OrderManagement.cls
public class OrderManagement {
    public static void createOrder(String productName, Integer quantity) {
        Order__c newOrder = new Order__c(Product_Name__c = productName, Quantity__c =
quantity);
        insert newOrder;
    }
}

// Apex Trigger: OrderTrigger.trigger
trigger OrderTrigger on Order__c (before insert) {
    for (Order__c order : Trigger.new) {
```

```
        if (order.Quantity__c <= 0) {
            order.addError('Quantity must be greater than zero');
        }
    }
}
```

Output:

Create a new order using the Apex class in Salesforce.

Trigger validation ensures quantity is greater than zero.

Successfully inserted orders are visible in Salesforce's Order object.

Result:

Successfully created an application in Salesforce.com using Apex, demonstrating custom business logic and validation.

4.Implementation of SOAP Web Services in C#/JAVA Applications

Aim:

To implement SOAP Web Services in a Java application, enabling data exchange over a network using XML-based protocols.

Procedure:

Step 1: Install necessary development tools (Visual Studio for C#, Eclipse/IntelliJ for Java)

Step 2: Create a SOAP web service with a WSDL file.

Step 3: Implement methods for data exchange.

Step 4: Deploy the SOAP web service and test it using a SOAP client.

Source code:

```
import javax.jws.WebMethod;
import javax.jws.WebService;
import javax.xml.ws.Endpoint;
```

```
@WebService
public class CalculatorService {

    @WebMethod
    public int add(int a, int b) {
        return a + b;
    }

    @WebMethod
    public int subtract(int a, int b) {
        return a - b;
    }

    public static void main(String[] args) {
        Endpoint.publish("http://localhost:8080/calculator", new CalculatorService());
        System.out.println("SOAP Web Service is running at http://localhost:8080/calculator");
    }
}
```

Output:

Access the service at:
http://localhost:8080/calculator?wsdl

Client requests can call methods like add and subtract using SOAP.

Result:

Successfully implemented a SOAP Web Service using Java, demonstrating XML-based communication for remote method invocation.

5.Implementation of Para-Virtualization using VMware/Oracle VirtualBox

Aim:

To implement Para-Virtualization using VMware/Oracle VirtualBox, demonstrating the efficient sharing of hardware resources and performance optimization.

Procedure:

Step 1: Install VMware Workstation or Oracle VirtualBox.

Step 2: Create a virtual machine and configure the guest OS.

Step 3: Enable para-virtualization settings.

Step 4: Run and test the virtualized environment.

Source code:

```
#!/bin/bash

# Script to automate para-virtualization setup on VirtualBox

VM_NAME="ParaVirtualVM"

ISO_PATH="/path/to/iso/file.iso"

VM_DISK="${VM_NAME}.vdi"

# Create VM

VBoxManage createvm --name "$VM_NAME" --register

# Set VM settings

VBoxManage modifyvm "$VM_NAME" --memory 2048 --cpus 2 --ostype Ubuntu_64

VBoxManage modifyvm "$VM_NAME" --nic1 nat --nictype1 virtio

VBoxManage modifyvm "$VM_NAME" --paravirtprovider kvm

# Create virtual disk and attach

VBoxManage createhd --filename "$VM_DISK" --size 20000
```

```
VBoxManage storagectl "$VM_NAME" --name "SATA Controller" --add sata --controller IntelAhci

VBoxManage storageattach "$VM_NAME" --storagectl "SATA Controller" --port 0 --device 0 --type hdd --medium "$VM_DISK"


# Attach ISO for installation

VBoxManage storageattach "$VM_NAME" --storagectl "SATA Controller" --port 1 --device 0 --type dvddrive --medium "$ISO_PATH"


# Start VM

VBoxManage startvm "$VM_NAME"
```

Output:

The para-virtualized VM efficiently shares resources, reducing overhead.

Improved I/O performance compared to fully virtualized VMs.

Result:

Successfully implemented Para-Virtualization using VMware/Oracle VirtualBox, demonstrating efficient hardware utilization and reduced overhead for guest OS.

6.Overview of Hadoop

AIM:

To provide an overview of Hadoop by demonstrating the creation of a simple file management system using HDFS (Hadoop Distributed File System).

Procedure:

Setup Hadoop Environment:

Install and configure Hadoop in standalone mode.

Start Hadoop services using:bash

start-dfs.sh start-yarn.sh

Create Directories in HDFS:

Use HDFS commands to create directories and manage files.

Upload, List, and Retrieve Files:

Store a file in HDFS and retrieve it using commands

Souce code:

```
# Step 1: Create Directory in HDFS
hdfs dfs -mkdir /user/hadoop_demo
echo "Directory /user/hadoop_demo created"

# Step 2: Upload a Local File to HDFS
echo "Hello, Hadoop World!" > local_file.txt
hdfs dfs -put local_file.txt /user/hadoop_demo/
echo "File uploaded to HDFS"

# Step 3: List Files in HDFS Directory
echo "Files in /user/hadoop_demo:"
hdfs dfs -ls /user/hadoop_demo

# Step 4: Retrieve File from HDFS
hdfs dfs -get /user/hadoop_demo/local_file.txt retrieved_file.txt
echo "File retrieved from HDFS: $(cat retrieved_file.txt)"
```

Output:

- Creates an HDFS directory /user/hadoop_demo.
- Uploads local_file.txt containing "Hello, Hadoop World!"
- Lists the file in HDFS:

pgsql

```
Found 1 items
-rw-r--r-- 1 hadoop hadoop 19 2025-04-03 10:00 /user/hadoop_demo/local_file.txt
```

- Retrieves the file from HDFS and displays content:

arduino

File retrieved from HDFS: Hello, Hadoop World!

Result:

Successfully demonstrated basic HDFS operations, providing a clear overview of Hadoop's distributed storage capabilities.

7. CREATING APPLICATION USING HADOOP MAPREDUCE

Aim:

To create a Hadoop MapReduce application that processes large datasets to count word occurrences, demonstrating distributed computing capabilities.

PROCEDURE:

Step 1: Write a Java-based MapReduce program (e.g., Word Count).

Step 2: Compile and package the application into a JAR file.

Step 3: Run the MapReduce job on a Hadoop cluster.

Step 4: Analyze the output results.

SOURCE CODE:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;


public class WordCount {

   public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {

      private final static IntWritable one = new IntWritable(1);
```

```
    private Text word = new Text();
    public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
        for (String token : value.toString().split(" ")) {
            word.set(token);
            context.write(word, one);
        }
    }
}

public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) sum += val.get();
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

```
}
```

OUTPUT:

hadoop 2

mapreduce 1

Result:

The MapReduce job efficiently processes large datasets to count word occurrences.

8. PLATFORM AS A SERVICE

Aim:

To deploy a dynamic web application on AWS using Elastic Beanstalk (PaaS), demonstrating easy scalability, management, and automation of web applications.

Procedure:

Develop a Node.js app using Express with a simple route to display a welcome message.

Initialize Elastic Beanstalk using eb init and create an environment with eb create.

Deploy the app using eb deploy and monitor the status.

Access the app using the Elastic Beanstalk environment URL.

Source code:

```
const express = require('express');

const app = express();

const port = process.env.PORT || 3000;


app.get('/', (req, res) => {

  res.send('<h1>Welcome to AWS PaaS Deployment!</h1><p>Your app is running on Elastic Beanstalk.</p>');

});
```

```
app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

Output:

The app displays:
Welcome to AWS PaaS Deployment!
"Your app is running on Elastic Beanstalk."

Result:

The Node.js app is successfully deployed on AWS Elastic Beanstalk using PaaS.

The application is accessible via a public URL, demonstrating automated scaling and management.

9.AMAZON WEB SERVICES

Aim:

To host a dynamic web application on Amazon Web Services (AWS) using EC2 for the backend server and S3 for static asset storage, demonstrating cloud scalability and availability.

Procedure:

Step 1: Study the AWS cloud architecture and services.

Step 2: Explore AWS EC2, S3, Lambda, and other services.

Step 3: Set up a basic web application using AWS services.

Step 4: Deploy and test the application on AWS.

Source Code:

```
import React from 'react';
import { Card, CardContent } from '@/components/ui/card';
import { motion } from 'framer-motion';
```

```
const App = () => {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-r from-blue-500 to-purple-600">
      <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ duration: 1 }}>
        <Card className="w-96 p-6 rounded-2xl shadow-lg">
          <CardContent>
            <h1 className="text-3xl font-bold text-center mb-4">Welcome to AWS Hosting!</h1>
            <p className="text-center">Your web app is successfully hosted on AWS.</p>
          </CardContent>
        </Card>
      </motion.div>
    </div>
  );
};
```

Output:

- Title: "Welcome to AWS Hosting!"
- Message: "Your web app is successfully hosted on AWS."

Result:

Successfully hosted on AWS, accessible via a public URL from S3 or EC2